

monitoring 설계·운영 매뉴얼

역할: FARM/LAB 서버와 사용자 GPU workload의 지속 관측, 시각화, 경보와 제한된 자동 복구를 소유한다.

1. 책임 범위

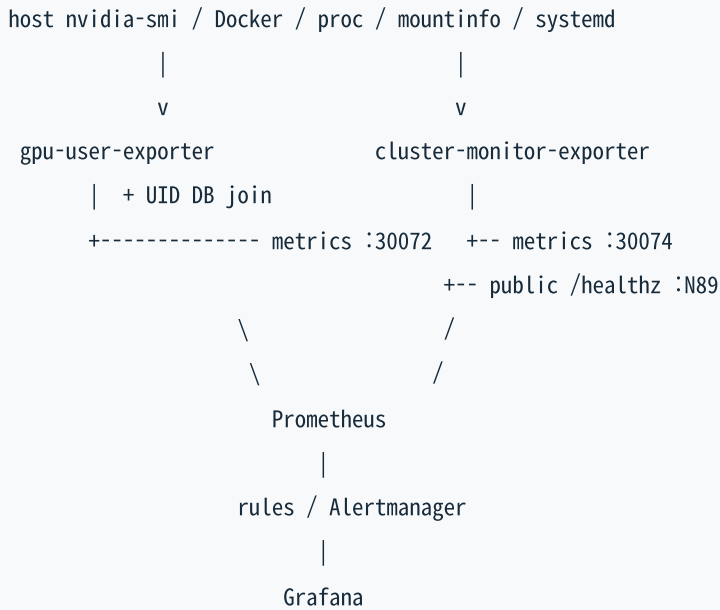
`monitoring`은 Kubernetes의 Prometheus/Grafana 자원과 각 GPU host에서 실행되는 exporter를 함께 관리한다. 서버가 켜진 직후 한 번 실행하는 점검은 `remote-operations`, 서버에 무엇이 설치되어야 하는지는 `server-state`가 소유한다. 이 디렉터리는 **실행 중인 시스템을 반복 관측하는 경로**다.

지속 관측과 부팅 작업을 분리한 이유는 일시적인 부팅 지연을 장기 장애로 오판하지 않고, exporter 재배포가 WOL이나 container restart 순서에 영향을 주지 않게 하기 위해서다.

2. 디렉터리 지도

경로	핵심 기능
<code>farm/prometheus/</code>	FARM kube-prometheus-stack values, PV, storage class
<code>lab/prometheus/</code>	독립 LAB Prometheus와 PV 설정
<code>farm/gpu/</code> , <code>lab/gpu/</code>	NFD와 NVIDIA device plugin 설정
<code>exporters/cluster-monitor-exporter/</code>	mount, GPU, Docker, container, 연결성, NFS GSS 지표
<code>exporters/gpu-user-exporter/</code>	GPU process를 UID DB의 실제 사용자/container에 귀속
<code>ansible_playbook/</code>	exporter, GSS health, NFS forensics 배포 entry point
<code>grafana/dashboards/</code>	FARM/LAB GPU, NIC, storage latency dashboard
<code>grafana/datasources/</code>	LAB Prometheus datasource override
<code>nfs-forensics/</code>	NFS incident용 제한된 packet/kernel trace ring
<code>shared/gpu-stress-test/</code>	공통 GPU test workload

3. 전체 데이터 흐름



FARM Grafana가 FARM과 LAB dashboard를 함께 제공하므로 dashboard는 cluster 하위가 아니라 공통 `grafana/`에 둔다. LAB dashboard는 UID가 `prometheus-lab` 인 datasource를 기대한다.

4. cluster-monitor-exporter

4.1 관측 대상

Go exporter는 host에서 다음 상태를 수집한다.

- 필요한 mount의 source/target 일치와 응답성
- storage peer 연결성과 mount failure 진단
- `/proc`의 blocked process와 NFS/RPC D-state
- host NVIDIA GPU 상태
- Docker daemon과 대상 container 상태
- container SSH/GPU readiness와 NVML mismatch
- 외부 인터넷/heartbeat 연결성
- Kerberos NFS readiness, canary와 bounded recovery 상태
- NFS 포렌식 snapshot 상태

`/metrics`는 마지막 collection 결과를 제공하고 `/healthz`는 collection이 최근에 성공했는지 확인한다. 별도 public health listener는 서버 번호별 예약 `N89` 포트로 외부에서 host reachability를 확인하게 한다.

4.2 자체 renderer와 명령 timeout

exporter는 수집 결과를 Prometheus text format으로 렌더링한다. 외부 명령은 timeout을 두고 실행한다. NFS `hard` mount 장애에서 child가 D-state가 되면 일반적인 context cancel만으로 종료되지 않을 수 있기 때문에, 수집 loop가 무한히 막히지 않도록 process와 결과 보존을 방어적으로 처리한다.

collection freshness도 health 판단에 포함한다. HTTP server가 응답한다는 사실만으로 수집이 정상이라고 보지 않기 위해서다.

4.3 제한된 복구

exporter는 설정에 따라 멈춘 대상 container 시작, container SSH 시작, 안전한 NVML library mismatch 복구처럼 범위가 제한된 작업을 할 수 있다. Kerberos NFS mount 복구는 별도 timer/script 상태를 관측하며 다음 gate를 통과한 missing mount만 복구한다.

1. fstab에 기대한 항목이 있다.
2. DNS와 storage RPC가 응답한다.
3. Kerberos readiness chain이 통과한다.
4. 해당 환경에서 요구되는 canary가 통과한다.
5. retry/inhibit 상태가 복구를 허용한다.

공유 NAS 서비스 재시작, 임의 unmount, user share 데이터 읽기는 exporter의 책임이 아니다. 자동 복구가 장애 원인보다 더 큰 장애 반경을 만들지 않게 하는 경계다.

5. gpu-user-exporter

5.1 해결하는 문제

`nvidia-smi` 는 GPU process와 PID를 보여 주지만 DECS 사용자 이름이나 DB의 container owner를 직접 알지 못한다. exporter는 다음 정보를 조합한다.

1. `nvidia-smi` 로 GPU device, process memory와 pmon utilization을 읽는다.
2. host `/proc/<pid>/cgroup` 에서 Docker container ID를 찾는다.
3. Docker inspect로 container 상태와 device 할당을 확인한다.
4. UID DB의 활성 container record를 cache하고 실제 사용자/username과 join한다.
5. 사용자·container·GPU별 memory, utilization, process count를 집계한다.

주요 지표는 `docker_gpu_user_memory_used_bytes` , `docker_gpu_user_sm_utilization_percent` , `docker_gpu_user_process_count` , device 전체 사용률과 exporter scrape/DB cache 상태다. DB에서 찾지 못한 process는 임의 사용자에게 귀속하지 않고 ignored count로 노출한다.

5.2 DB cache 설계

매 scrape마다 DB와 Docker에 과도한 부하를 주지 않도록 활성 container owner 목록을 일정 간격으로 cache한다. DB refresh 성공 여부와 cache entry 수를 별도 지표로 내보내므로 stale mapping 가능성을 운영자가 확인할 수 있다. DB 비밀번호는 배포 시 local env에서 target systemd 환경 파일로 전달하며 소스나 dashboard에 넣지 않는다.

6. Prometheus와 Grafana

FARM

- `kube-prometheus-stack` 을 사용한다.
- Prometheus/Grafana local PV는 관리 desktop의 `/data` 경로를 사용한다.
- Grafana NodePort 기본은 `30080` 이다.
- FARM/LAB datasource와 dashboard를 한 Grafana에서 제공한다.

LAB

- LAB8 control plane의 독립 cluster로 운영한다.
- LAB Prometheus는 기본적으로 cluster 내부 service다.
- FARM Grafana가 질의해야 할 때만 검토된 routed endpoint를 제공한다.

cluster별 values를 나눈 이유는 target, retention, storage, Alertmanager routing과 control plane 장애가 독립적이기 때문이다. dashboard는 공통 UI이므로 한 곳에 두되 datasource UID와 label로 FARM/LAB를 분리한다.

7. NFS GSS health와 forensics

Readiness와 canary

GSS health 배포는 다음 순서를 검증한다.

```
keytab -> kinit -> NFS service kvno -> rpc_pipefs -> rpc-gssd -> canary
```

부팅 시 mount가 host identity 준비보다 먼저 실행되는 race를 막기 위해 단계별 결과를 상태 파일에 기록한다. canary는 실제 user share가 아니라 전용 read-only 경로를 사용하는 것이 원칙이다. 전용 export가 없는 FARM은 canary gate를 비활성화하되 나머지 readiness는 유지한다.

Passive forensics

`nfs-forensics` 는 TCP/2049 packet의 크기 제한 ring과 저용량 ftrace instance, 5초 간격 상태 관측을 유지하다 incident threshold에서 snapshot을 보존한다. 다음 작업은 하지 않는다.

- mount/unmount
- 사용자 share 읽기
- NFS/GSS service restart
- recovery 또는 reboot

`sec=krb5` 는 payload를 암호화하지 않으므로 capture와 incident directory는 root-only로 유지한다. exporter는 `/run/decs-nfs-forensics/status.json` 을 읽을 뿐 capture service를 제어하지 않는다. 관측 경로가 장애 복구와 결합되어 증거를 파괴하지 않게 하기 위한 설계다.

8. 배포

두 exporter를 한 host에 배포:

```
cd /home/jy/server_manage/monitoring
ANSIBLE_CONFIG=ansible_playbook/ansible.cfg \
ansible-playbook ansible_playbook/deploy_exporters.yml \
-e exporter_hosts=farm8
```

cluster-monitor exporter만 전체 배포:

```
ANSIBLE_CONFIG=ansible_playbook/ansible.cfg \
ansible-playbook ansible_playbook/deploy_cluster_monitor_exporter_all.yml
```

NFS GSS health를 제한된 host에 배포:

```
ANSIBLE_CONFIG=ansible_playbook/ansible.cfg \
ansible-playbook ansible_playbook/deploy_nfs_gss_health.yml \
-e nfs_gss_health_hosts=lab8
```

NFS forensics 배포:

```
ANSIBLE_CONFIG=ansible_playbook/ansible.cfg \
ansible-playbook ansible_playbook/deploy_nfs_forensics.yml
```

playbook은 controller에서 Go binary를 build하고 systemd env/service를 target에 설치한 뒤 local HTTP endpoint를 검증한다. 새 서버는 Ansible inventory, Prometheus scrape target, 공개 health port와 dashboard label을 모두 확인한다.

9. 테스트와 검증

```
cd /home/jy/server_manage/monitoring

go test ./exporters/cluster-monitor-exporter/cmd/cluster-monitor-exporter/...
go test ./exporters/gpu-user-exporter/...
bash nfs-forensics/tests/test_watch.sh
bash exporters/cluster-monitor-exporter/tests/test_nfs_gss_ready.sh
bash exporters/cluster-monitor-exporter/tests/test_nfs_gss_mount_recovery.sh
```

배포 후 최소 확인 항목:

- `:30072/-/healthy` 와 `:30072/metrics`
- `:30074/healthz` 와 `:30074/metrics`
- public `N89/healthz`
- Prometheus target의 up 상태와 scrape freshness
- Alert rule label이 Alertmanager route와 일치하는지
- FARM/LAB dashboard가 올바른 datasource를 보는지

10. 변경 가이드와 안전 수칙

- 새 metric은 이름, help, label cardinality와 실패 시 값을 함께 설계한다.
- container ID, PID처럼 cardinality가 큰 label은 필요성을 검토한다.
- 자동 복구는 사전조건, timeout, retry limit와 inhibit 상태를 가져야 한다.
- NFS probe는 user data를 읽지 않는 경로와 bounded timeout을 사용한다.
- exporter binary와 runtime incident data를 Git에 추가하지 않는다.
- local datasource credential과 DB password는 example 파일에 실제 값으로 넣지 않는다.
- 지속 monitor에서 새로 발견한 high-risk 복구는 `kerberos-nfs` 또는 소유 모듈의 명시적 runbook으로 분리한다.